BART DE MOT · OCTOBER 2026

Can a neural network beat a golden template?

A learning study on public inspection images

I spent 18 years around inspection equipment, where finding the part that does not look like the good ones is done with golden templates and a lot of tuning. This study asks how far a neural network gets from good images only, and what holds up when the images change.

Short answer. On four of six categories, pretrained features beat my golden template clearly, but the reject line has to be re-checked whenever lighting or focus changes.

How the inspection works

1 Train
Show the method good parts only. Defects are too rare to learn from.

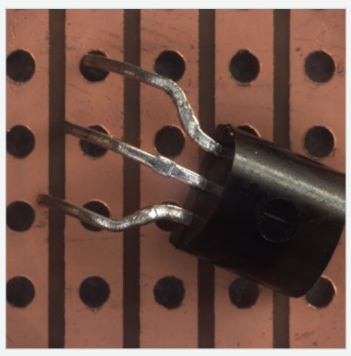
2 Score
Each new part gets a suspicion score: how unlike the good parts does it look?

3 Set a line
Choose a score above which a part is rejected. This is the threshold.

4 Decide
Below the line the part passes. Above it, the part is rejected.

What is inspected

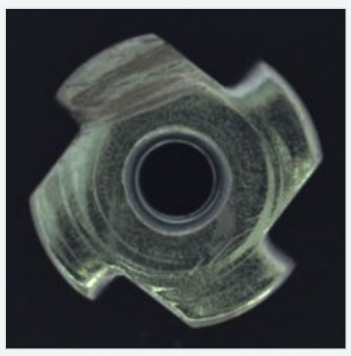
Six of the fifteen MVTec AD categories: three objects (transistor, pill, metal nut) and three textures (leather, carpet, grid). Each image shows an example defect.



Transistor
a misplaced transistor



Pill
a crack



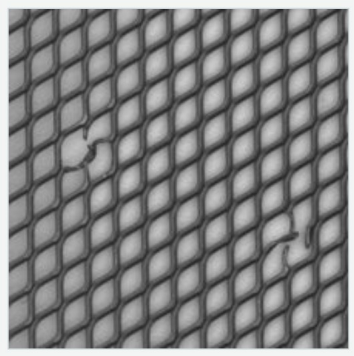
Metal nut
a scratch



Leather
a cut



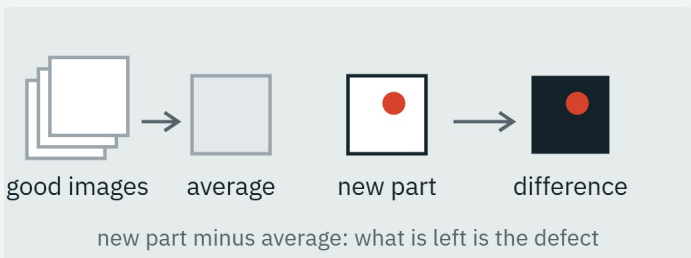
Carpet
a hole



Grid
a broken wire

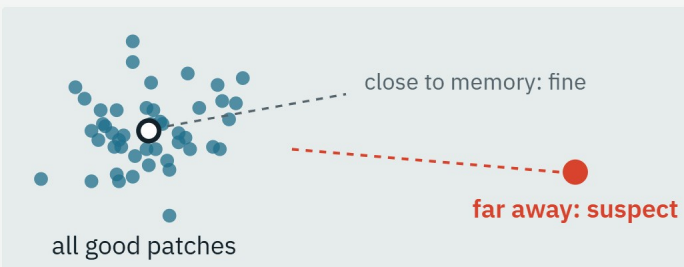
Three methods

Golden template



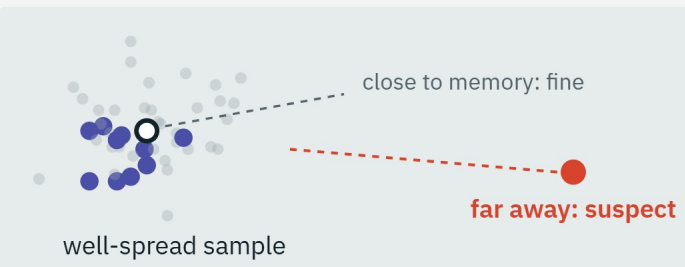
Average many good images into one picture. Compare each new part with it; where they differ, there is a defect. The classical approach I know.

Nearest neighbour



A pretrained network turns each small patch of a good image into a point. A new patch far from every remembered point is suspect.

PatchCore



The same idea, but it keeps only a small, well-spread sample of the good patches instead of all of them. Best overall, at about fifty times the fitting time of nearest neighbour.

Limits Six of fifteen categories; robustness results shown for three objects; small test sets; mostly one seed, CPU only; changes are simulated; built with an AI assistant. Notebooks available on request.

Images: MVTec AD, CC BY-NC-SA 4.0 (Bergmann, Fauser, Sattlegger and Steger, CVPR 2019). PatchCore: Roth et al., CVPR 2022.

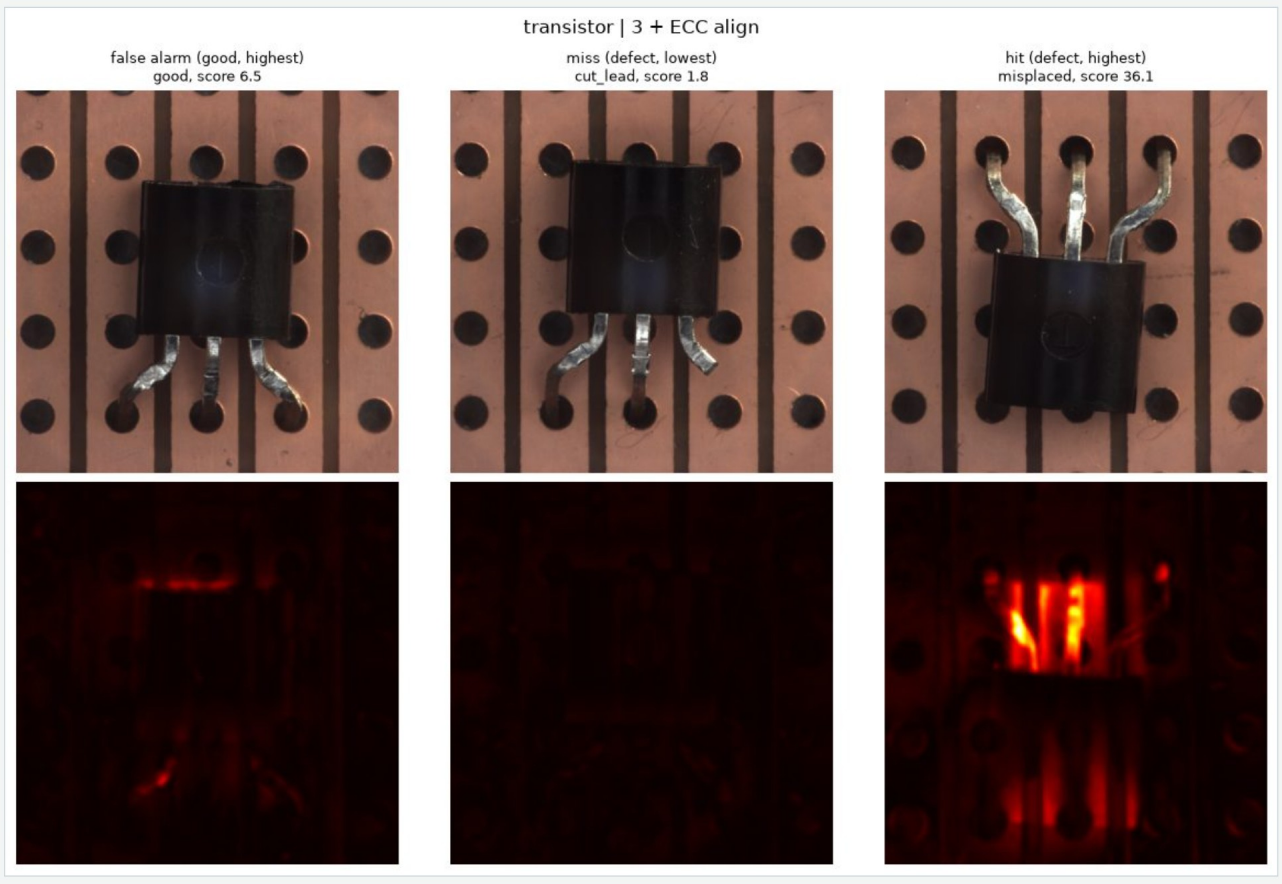
BART DE MOT · OCTOBER 2026

Catching a defect, missing one, a false alarm

The same transistor board seen by two methods

Bright areas in the lower pictures are where the method sees something unlike the good parts. Examples are single test images chosen to show each case, not typical rates. PatchCore uses the same patch idea as nearest neighbour with a smaller memory.

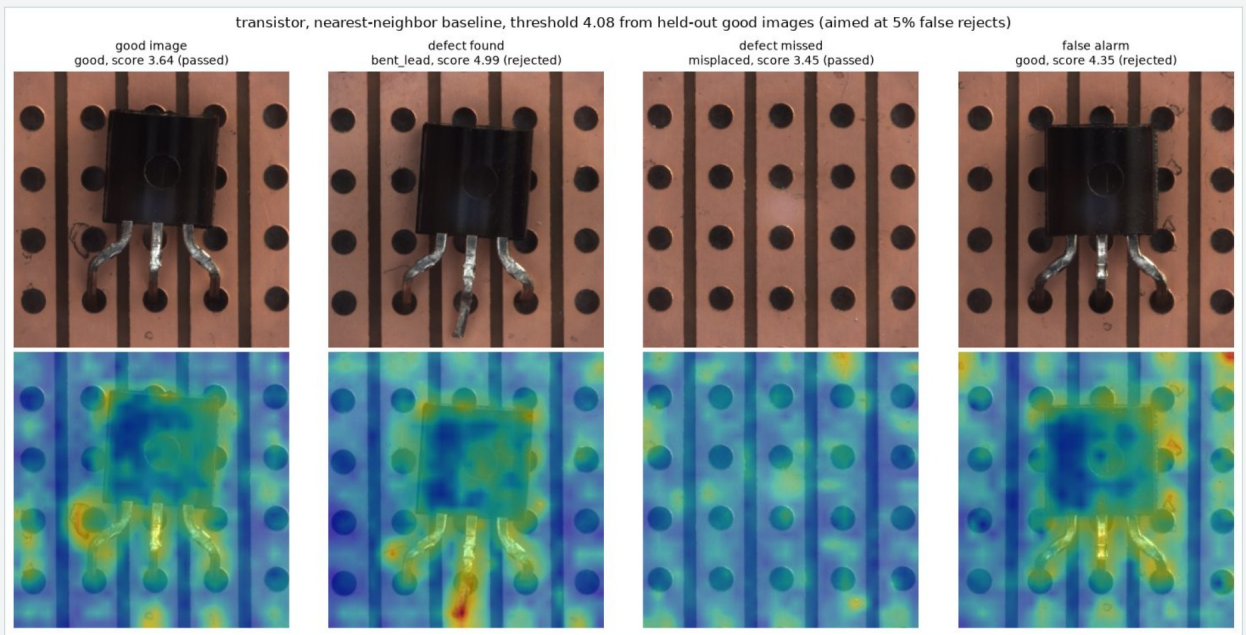
Golden template



Top: the part. Bottom: how much each pixel differs from the aligned template.

- Right: a misplaced transistor lights up strongly (score 36.1).
- Middle: a cut lead is almost invisible to the template (1.8).
- Left: a good part still shows some difference (6.5), the template's version of a false alarm.

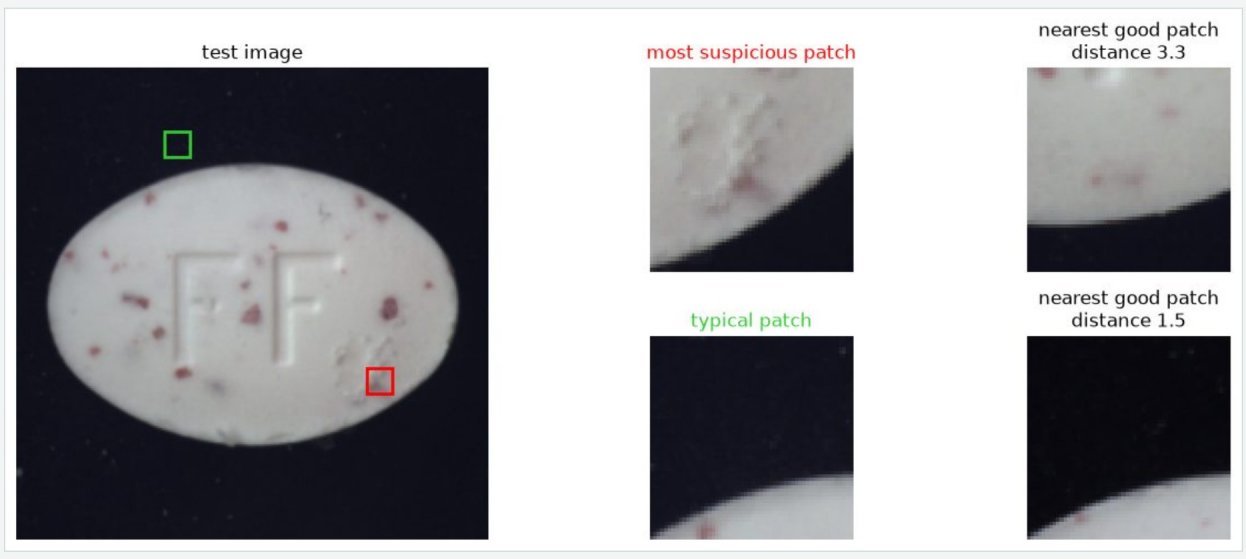
Nearest neighbour



Heat map: red means the patch is far from anything in the memory of good parts. The reject line was set from held-out good images, aiming at 5% false rejects.

- A bent lead is found (4.99, above the 4.08 line).
- A missing transistor is missed (3.45): the heat map shows nothing unusual.
- A good part is rejected (4.35): a false alarm.

One patch up close



On a pill, the most suspicious patch (red) is compared with the closest patch in the memory of good images. Smaller distance means more normal.

- Suspicious patch: nearest good patch is 3.3 away.
- Typical patch (green): nearest good patch is only 1.5 away.

Images: MVTec AD, CC BY-NC-SA 4.0 (Bergmann, Fauser, Sattlegger and Steger, CVPR 2019).

BART DE MOT · OCTOBER 2026

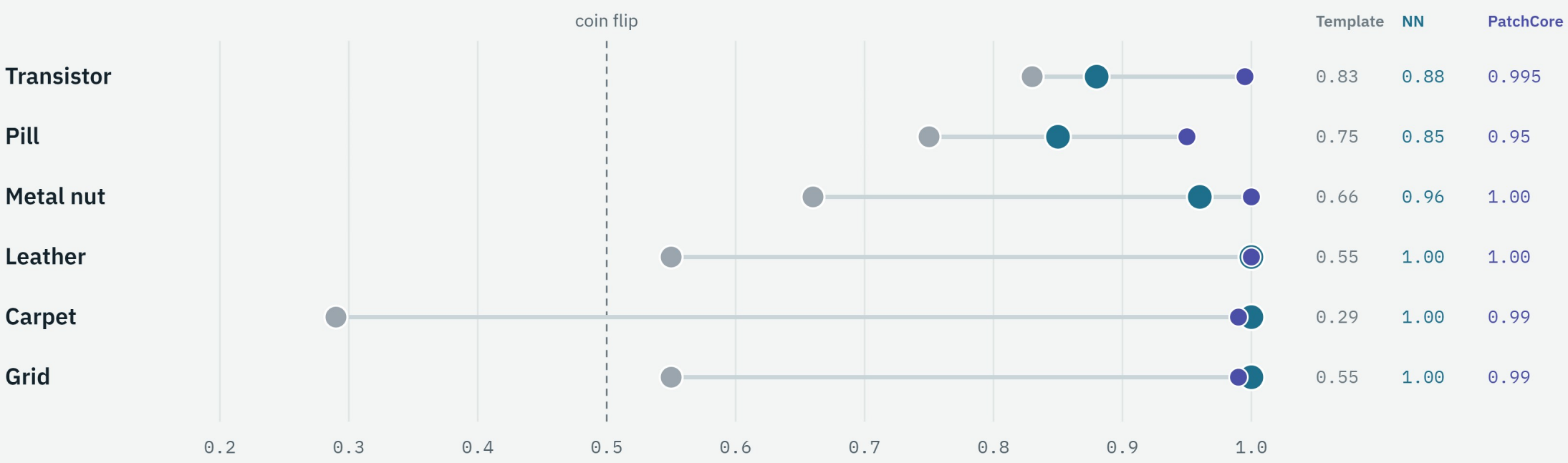
Neural methods win, but the threshold drifts

Image AUROC per category, then 14 simulated changes

AUROC is how often a defective part scores as more suspicious than a good one: 1.0 is always, 0.5 a coin flip. It measures ranking, not where the reject line sits. A false reject is a good part wrongly flagged. All methods run on a CPU; neural results in the last chart are a mean of five seeds, the rest use one.

How well do they separate good from defective?

Nearest neighbour is shown with a border margin ignored; a tuned classical variant reaches 0.99 on leather and 0.95 on grid.



4 of 6
categories where pretrained features beat the template clearly. On transistor and pill the simple neural method is not clearly better; PatchCore is.

0.77 → 1.00
grid, by ignoring a border artifact. Cost: one of 57 defective grid images, with its defect inside the border, can no longer be found.

up to 36%
real false-reject rate against a 5% target (PatchCore, 14 to 36% on five of six).

What happens when the images change?

Changes move the reject line far more than the ranking. Each method has a weak spot, the template's is lighting, and with that fixed the neural method still finds more defects. 14 simulated changes on three objects (transistor, pill, metal nut), thresholds fixed beforehand.

Golden template

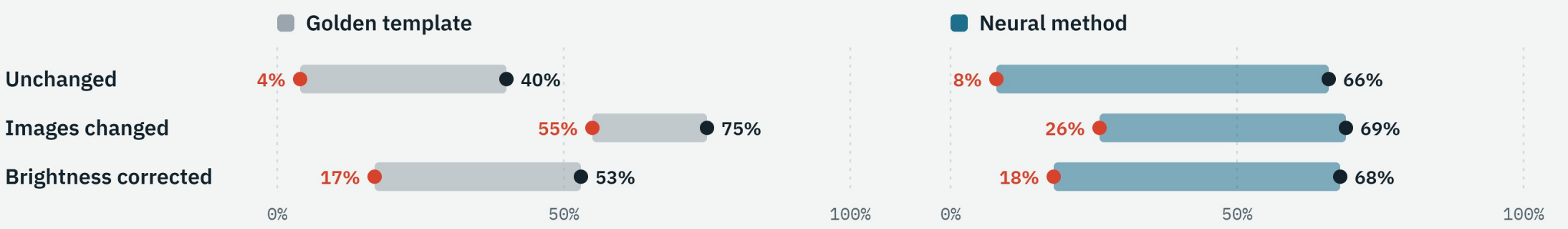
Needs	a stable light
Half-darker light	rejects 100% of good parts (15% darker: 44%)
Ranking	AUROC falls from 0.74 to 0.45
Threshold set again	11% rejected, 34% found

Neural method

Needs	a sharp, aligned image
Half-darker light	rejects 3% of good parts
Ranking	AUROC 0.89 to 0.84 on average, 0.76 with blur
Threshold set again	7% rejected, 56% found; less for blur, rotation, shift

Is the comparison fair to the template?

With the brightness fix the template wins back all of its good-from-defective separation and the neural method most of it, and the neural method stays ahead.



Limits

Data	Small test sets: 0.01 to 0.02 is noise. The best classical variant and network layer were picked on the test images.
Changes	Simulated, not real lots or lamps. Refitting the neural memory on changed images fixes most false rejects for blur and lens dirt, at 2 to 10 times the fitting time; only tested on seen changes.
Models	PatchCore needs about 0.4 s per image on a CPU. The patch methods miss misplaced transistors.
Production	Good enough to learn from, not to run a line: that needs stable imaging, far more real good parts and real defects.

Images: MVTec AD, CC BY-NC-SA 4.0 (Bergmann, Fauser, Sattlegger and Steger, CVPR 2019).

BART DE MOT · OCTOBER 2026

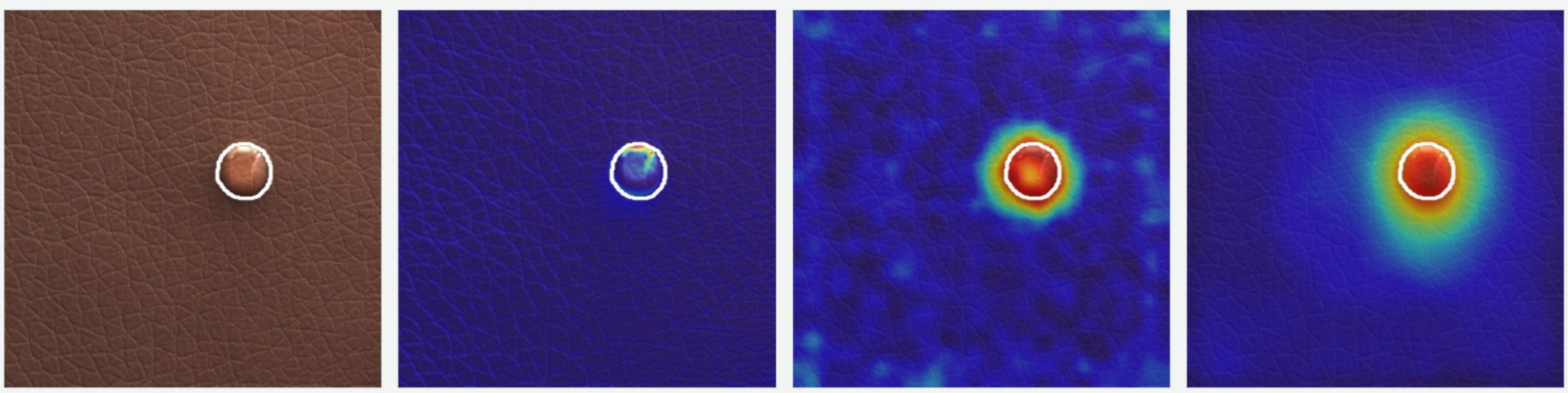
Where does each method look?

An easy case, a hard case, and how well the scores separate

The white outline is the defect as marked in the dataset. Red is the most unusual spot in that image, with each map scaled to its own image rather than to a fixed score. The examples are single test images chosen to show each case, not typical rates; the score plot uses only a few test images per category.

An easy case: glue on leather

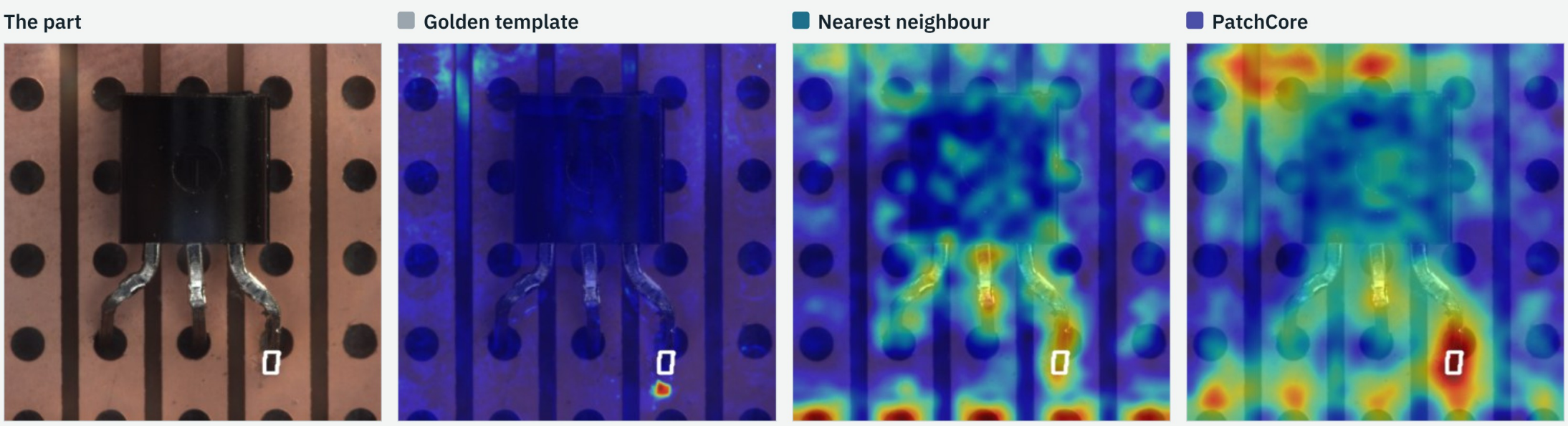
All three methods find it; they differ in how much of it they mark.



Nearest neighbour and PatchCore both mark the whole glue spot and little else. The template marks only part of it.

A small defect: a bent lead

The hard case: the outline is a few pixels, and the maps show other bright spots too.



PatchCore's brightest region covers the defect, next to another bright patch at the top left. The template shows a small hot spot just beside the outline. Nearest neighbour lights up many places and does not single out the defect.

Does the best method reject the defective parts?

The heat maps show where; this shows the decision. PatchCore, the best method, on two good and the defective test images per category.



On leather the two groups are far apart (good 26 to 29, defective 50 to 85). On transistor the gap shrinks: among the images plotted, the lowest defective image (the bent lead above, 38.5) is only 2.6 points above the highest good one (35.9). Over the whole test set the two groups overlap (PatchCore fitted on 80% of the good images): good transistor images reach 45.5 and the lowest defective score is 36.1.

Images: MVTec AD, CC BY-NC-SA 4.0 (Bergmann, Fauser, Sattlegger and Steger, CVPR 2019).